

Adversarial Search

Serkan Ayvaz

Games and adversarial search



What kind of Environment?

- **Competitive** and **Multiagent** environment, where other agents are plotting against us!
- We will cover games like Chess, Go and Poker

Why study games?

- Traditional hallmark of intelligence
- Easy to formalize
- Good model of real-world competitive activities
 - Military confrontations, negotiation, auctions, etc.

Types of game environments

	Deterministic	Stochastic
Perfect information (fully observable)	Chess, checkers, go	Backgammon, monopoly
Imperfect information (partially observable)	Battleships	Scrabble, bridge

Alternating two-player zero-sum games

- Players take turns
- Zero-sum: what is good for one player is just as bad for the other:
no win-win!
- **Utility** for each player (e.g., 1 for win, 0 for loss) for each game outcome or **terminal state**
- Constant sum of both players' utilities

Games vs. single-agent search

We don't know how the opponent will act

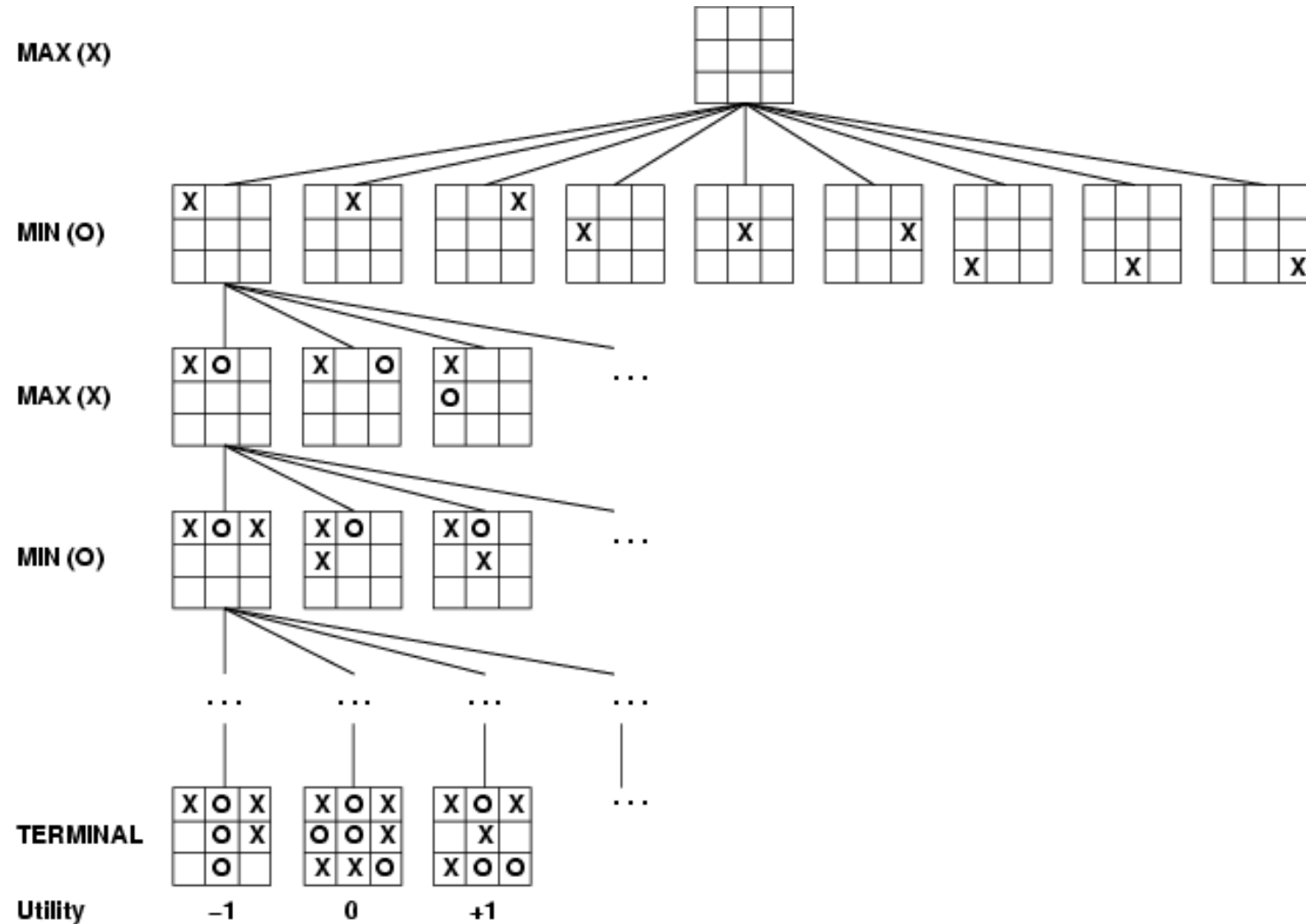
- Solution: ***strategy*** or ***policy*** (a mapping from state to best move in that state)

Efficiency is critical to playing well

- Limited time to make a move
- Huge branching factor, search depth, and number of terminal configurations
 - In chess, branching factor ≈ 35 and depth ≈ 80 ply, giving a search tree of $35^{80} \approx 10^{123}$ nodes
- Searching all the way to the end of the game – ruled out

Game tree

A game of tic-tac-toe between two players, “max” and “min”

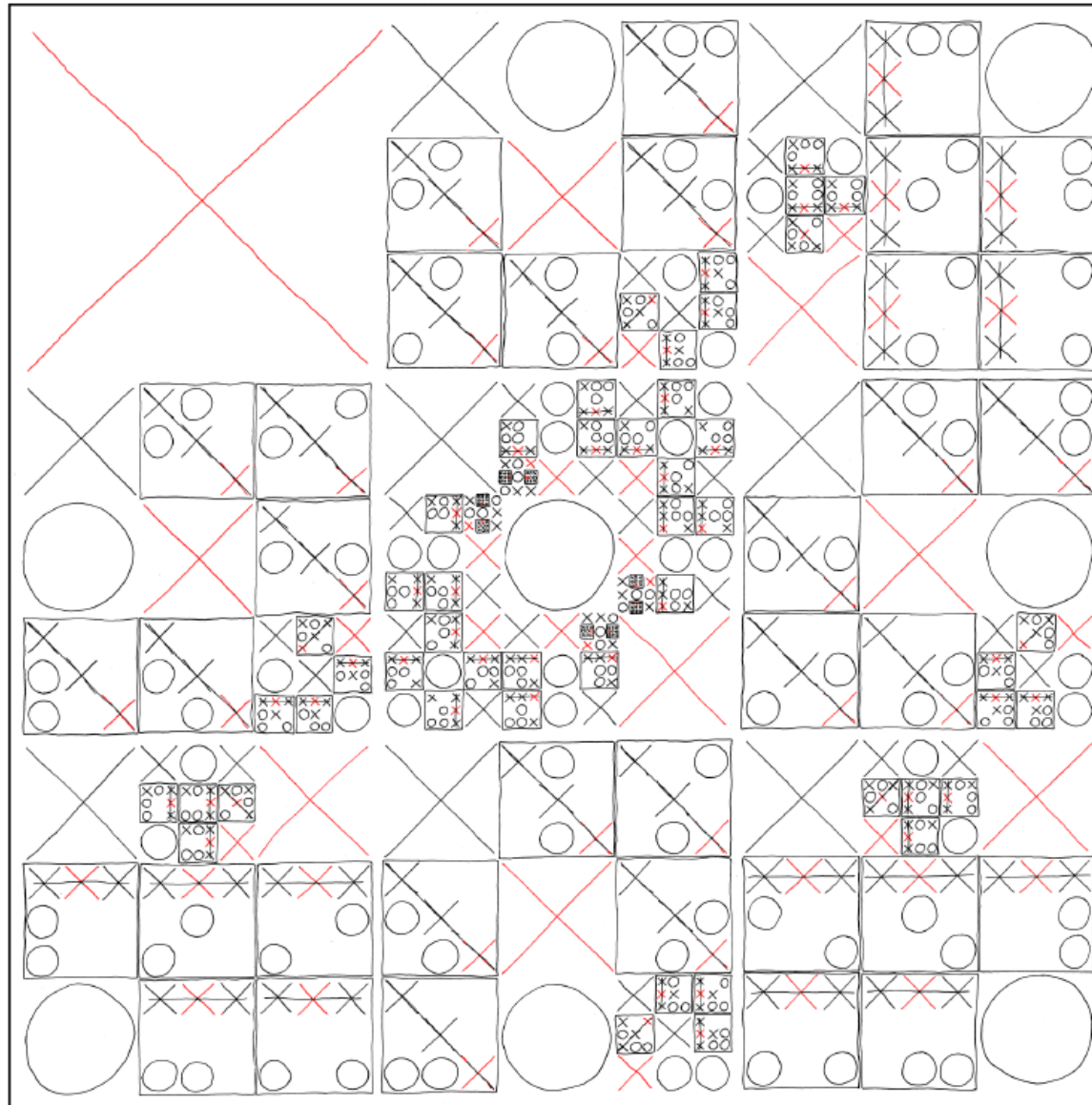


COMPLETE MAP OF OPTIMAL TIC-TAC-TOE MOVES

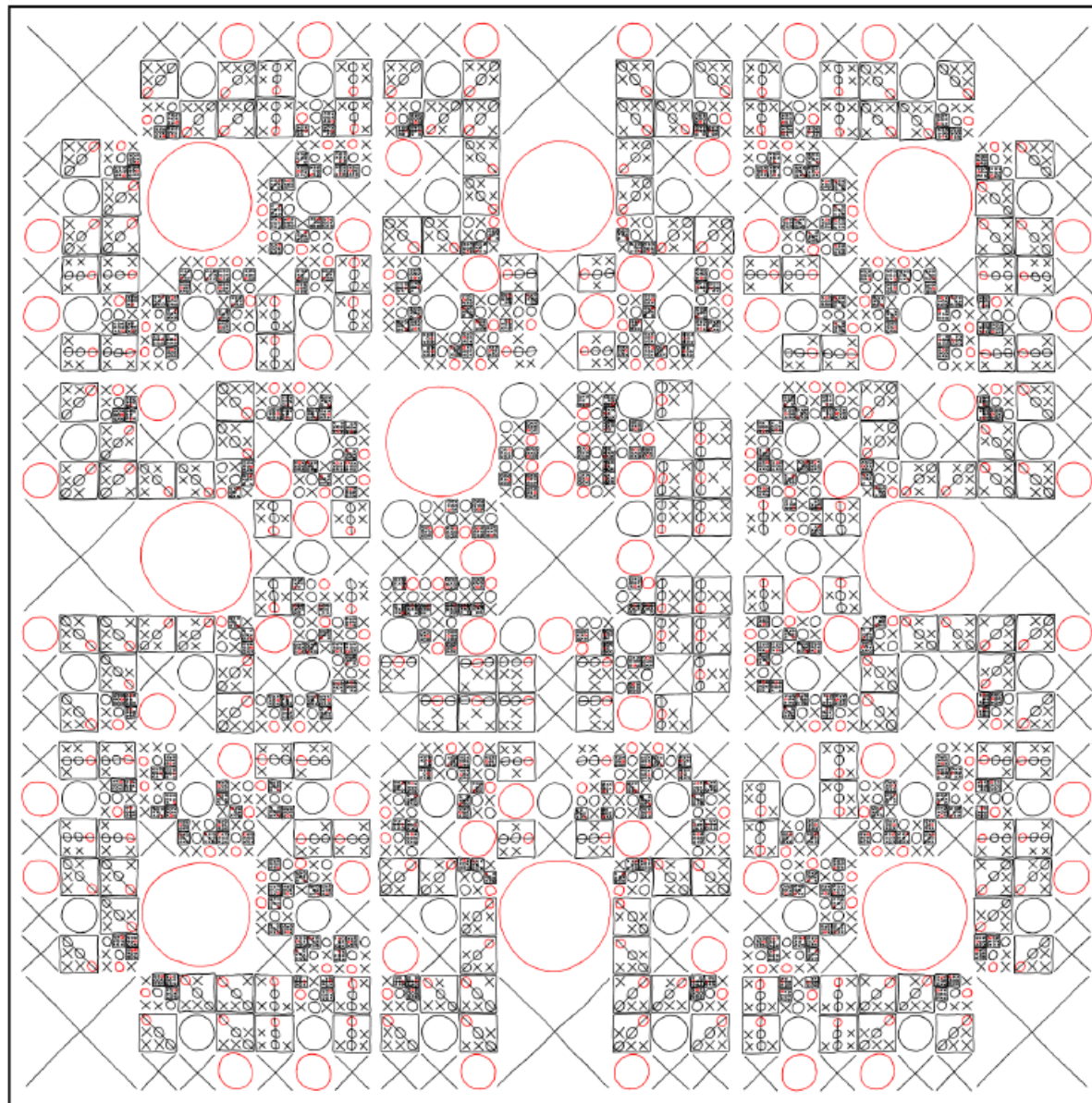
YOUR MOVE IS GIVEN BY THE POSITION OF THE LARGEST RED SYMBOL ON THE GRID. WHEN YOUR OPPONENT PICKS A MOVE, ZOOM IN ON THE REGION OF THE GRID WHERE THEY WENT. REPEAT.

<http://xkcd.com/832/>

MAP FOR X:

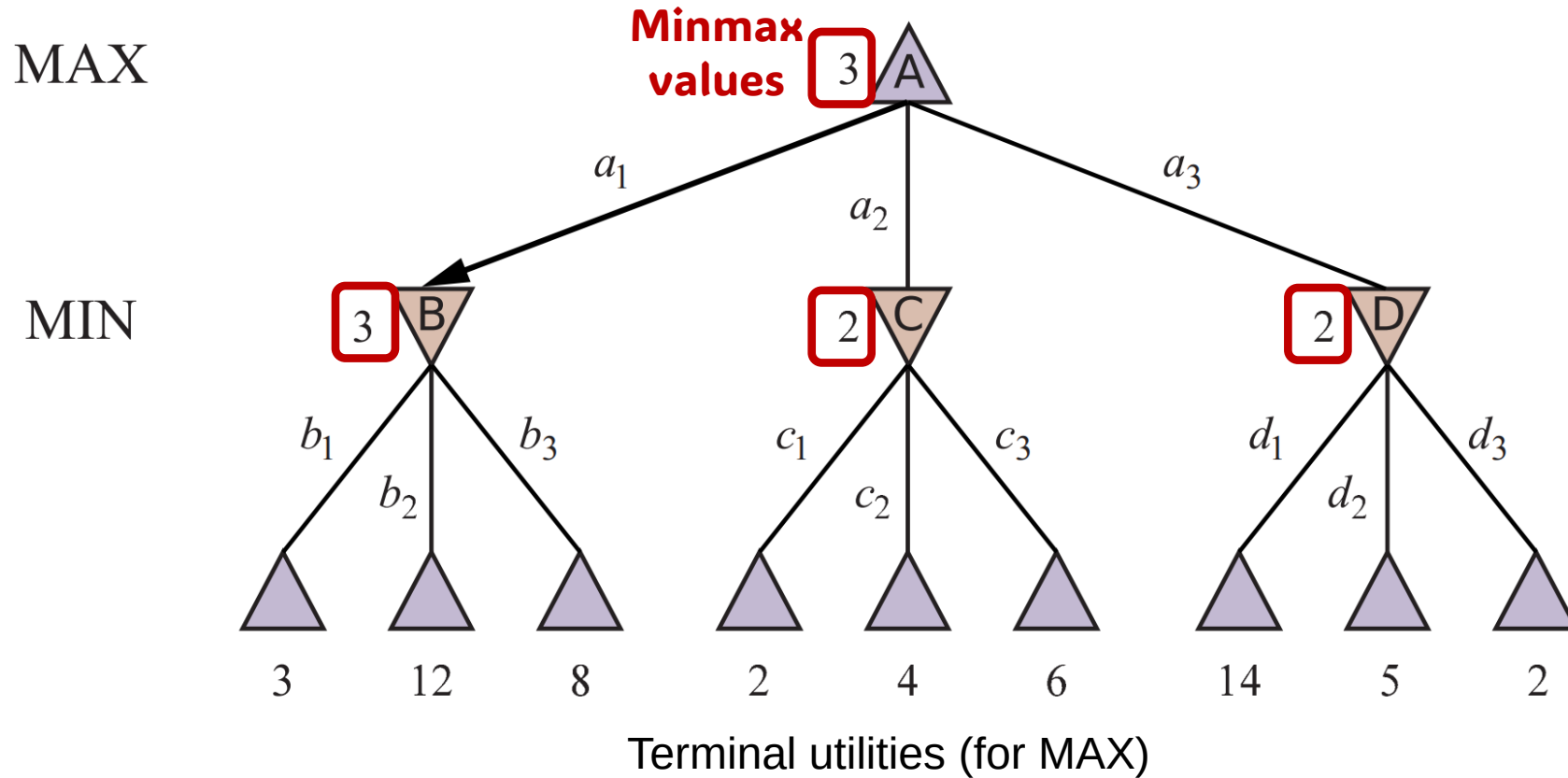


MAP FOR O:



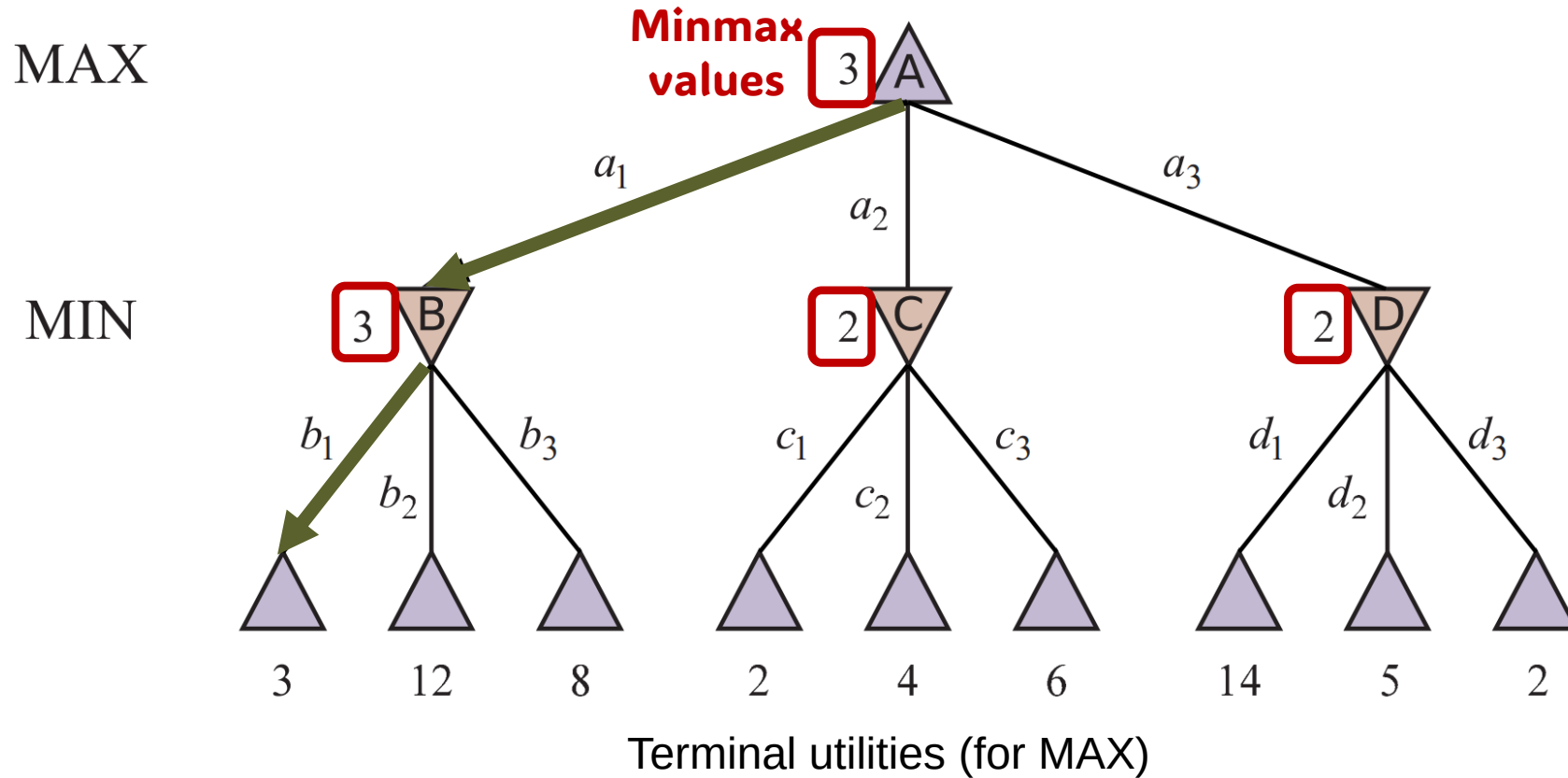
<http://xkcd.com/832/>

A more abstract game tree



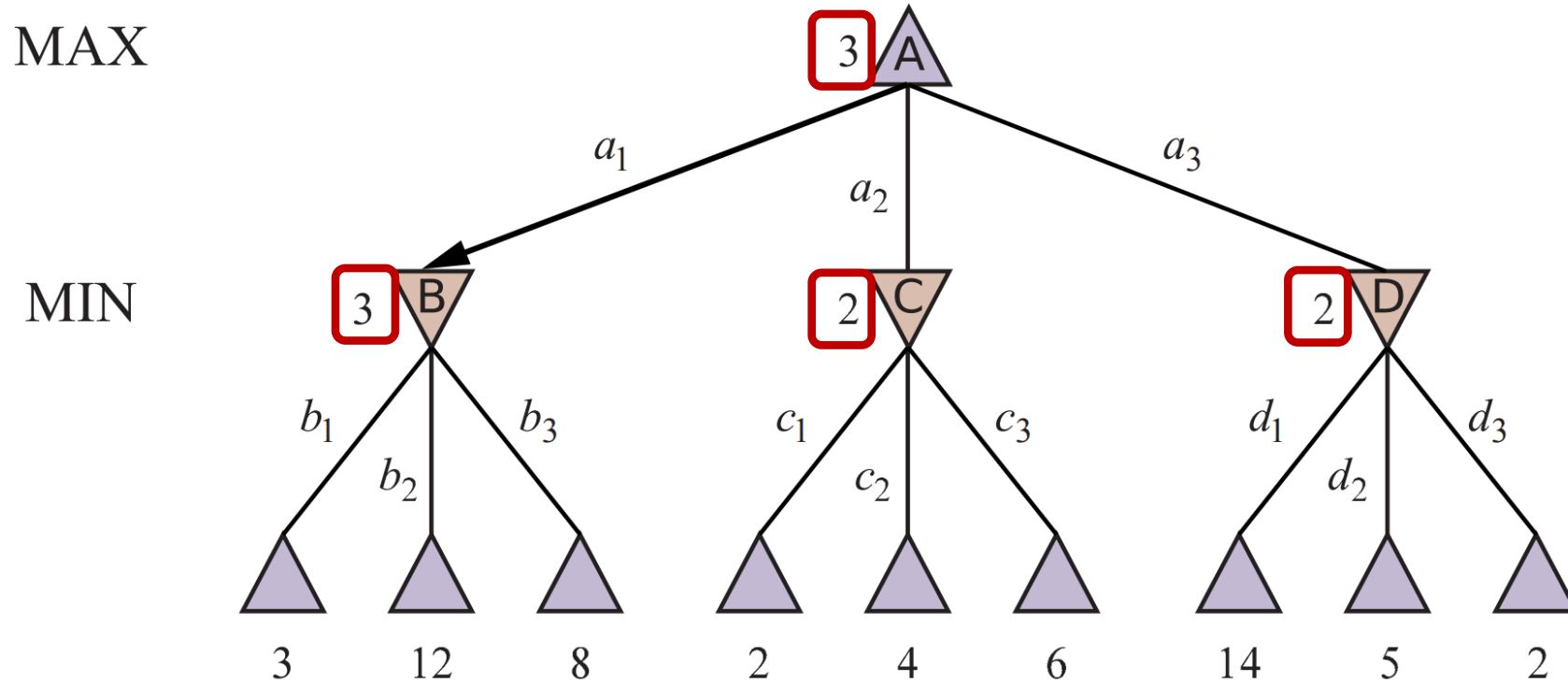
A two-player game

A more abstract game tree



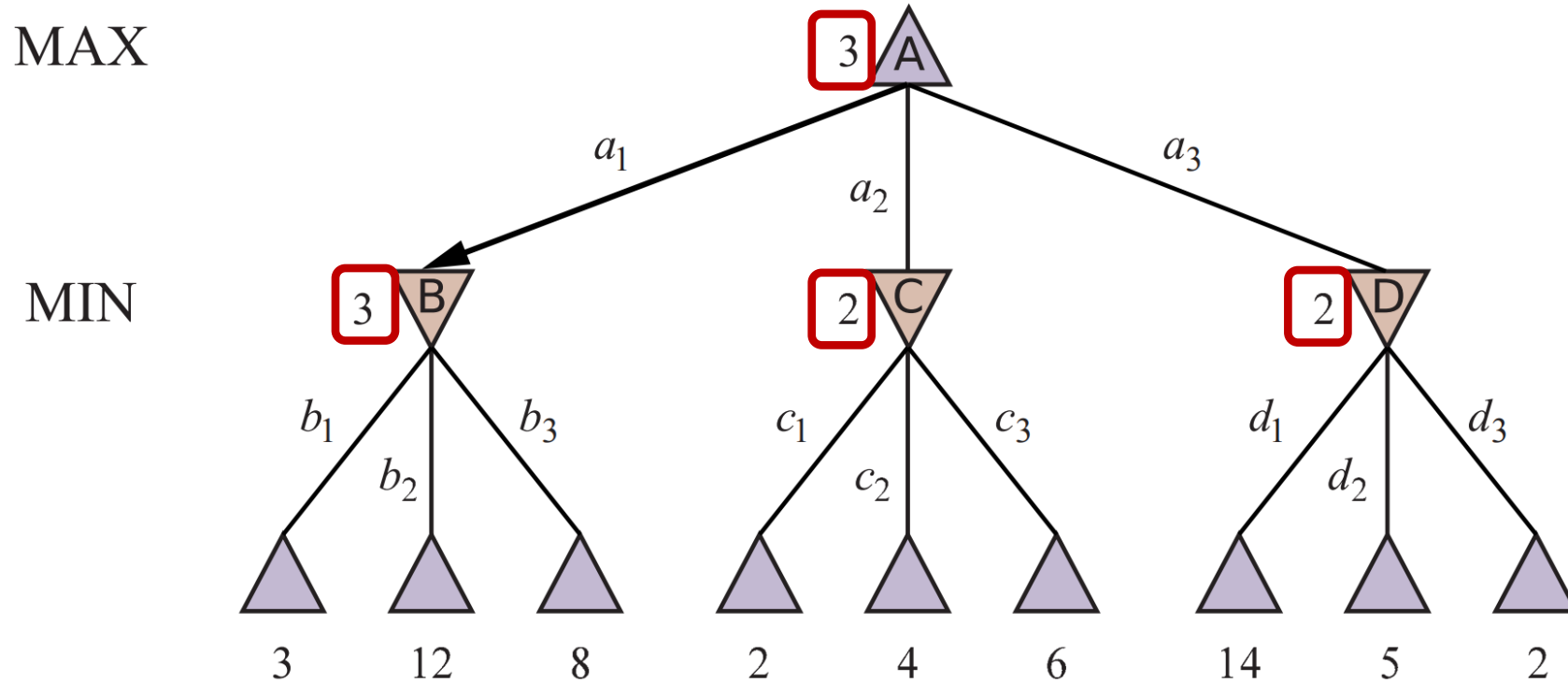
A two-player game

A more abstract game tree



- **Minimax value of a node:** the utility (for MAX) of being in the corresponding state, assuming perfect play on both sides
- **Minimax strategy:** Choose the move that gives the best worst-case payoff

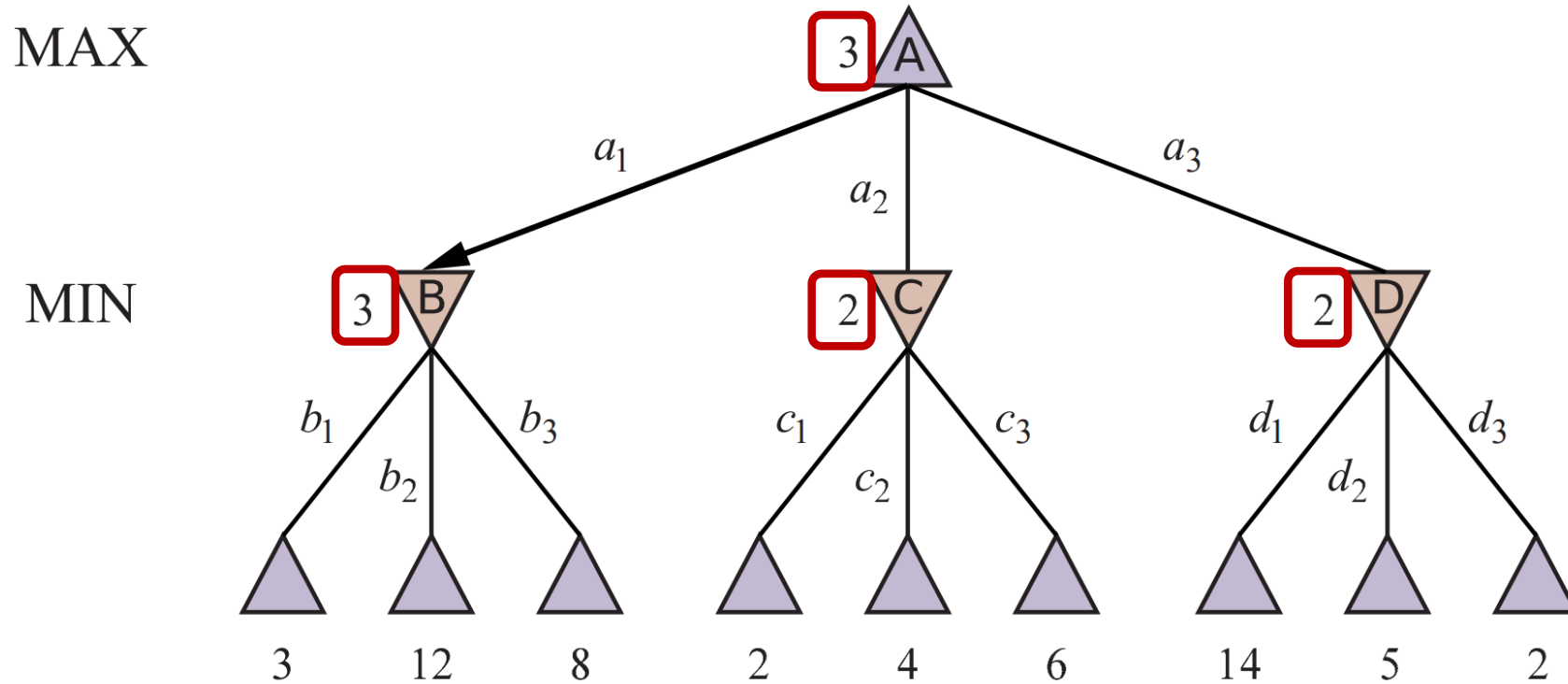
Computing the minimax value of a state



Minimax(state) =

- Utility(state) if state is terminal
- max **Minimax**(successors(state)) if player = MAX
- min **Minimax**(successors(state)) if player = MIN

Computing the minimax value of a state



The minimax strategy is optimal against an optimal opponent

- If the opponent is sub-optimal, the utility can only be higher
- A different strategy may work better for a sub-optimal opponent, but it will necessarily be worse against an optimal opponent

Example

Coins game

- There is a stack of N coins
- In turn, players take 1, 2, or 3 coins from the stack
- The player who takes the last coin loses

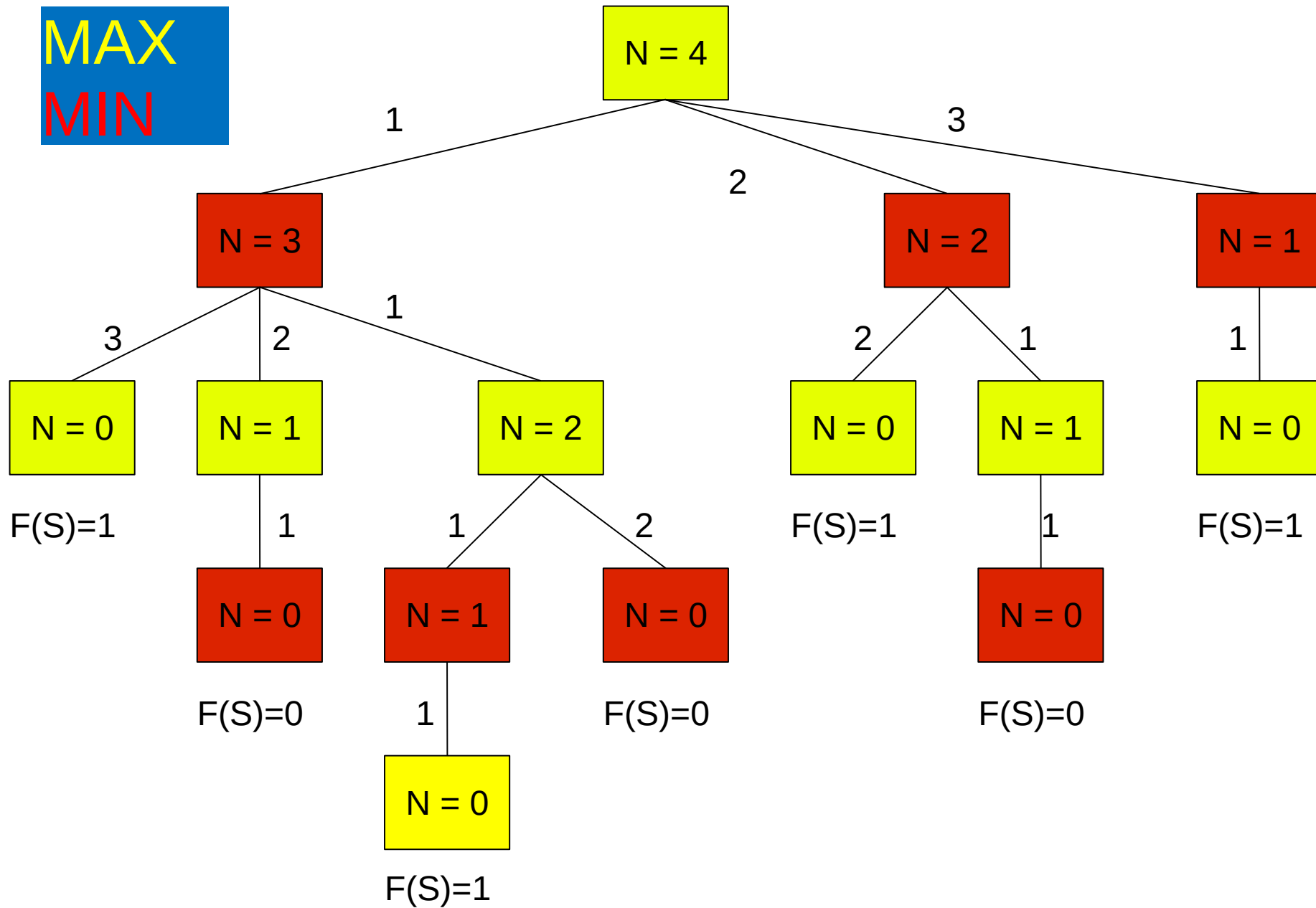


Coins Game: Formal Definition

- **Initial State:** The number of coins in the stack
- **Operators:**
 1. Remove one coin
 2. Remove two coins
 3. Remove three coins
- **Terminal Test:** There are no coins left on the stack
- **Utility Function:** $F(S)$
 - $F(S) = 1$ if MAX wins, 0 if MIN wins



MAX
MIN



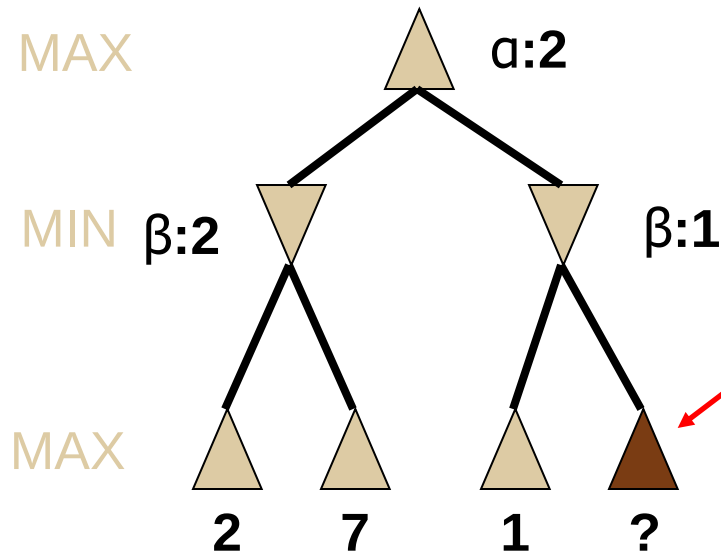
Analysis

- Max Depth: 5
- Branch factor: 3
- Number of nodes: 15
- Even with this trivial example, you can see that these trees can get very big
 - Generally, there are $O(b^d)$ nodes to search for
 - ❖ Branch factor b : maximum number of moves from each node
 - ❖ Depth d : maximum depth of the tree
 - Exponential time to run the algorithm!
 - How can we make it faster?

Alpha-Beta Pruning

Improve performance of the minimax algorithm through alpha-beta pruning

“If you have an idea that is surely bad, don't take the time to see how truly awful it is” - Pat Winston (MIT)



- We don't need to compute the value at this node
- No matter what it is, it can't affect value of the root node

Alpha-Beta Pruning



Main idea: Avoid processing subtrees that have no effect on the result



Two new parameters

α : The best value (highest) for MAX seen so far
 β : The best value (lowest) for MIN seen so far



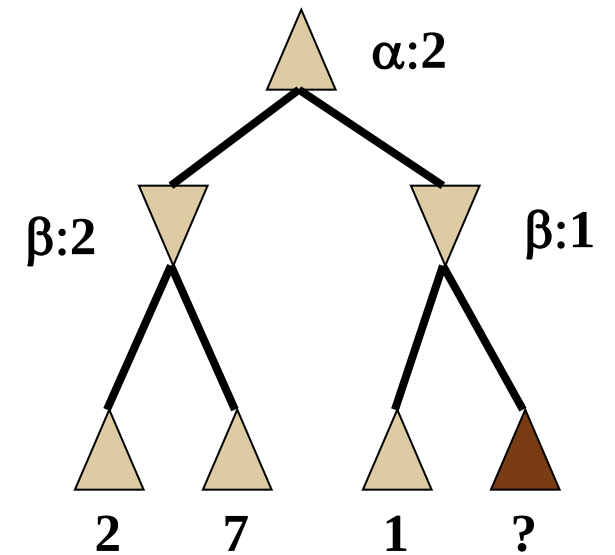
α is used in MIN nodes, and is assigned in MAX nodes (Think: α = “at least”)



β is used in MAX nodes, and is assigned in MIN nodes (Think: β = “at most”)

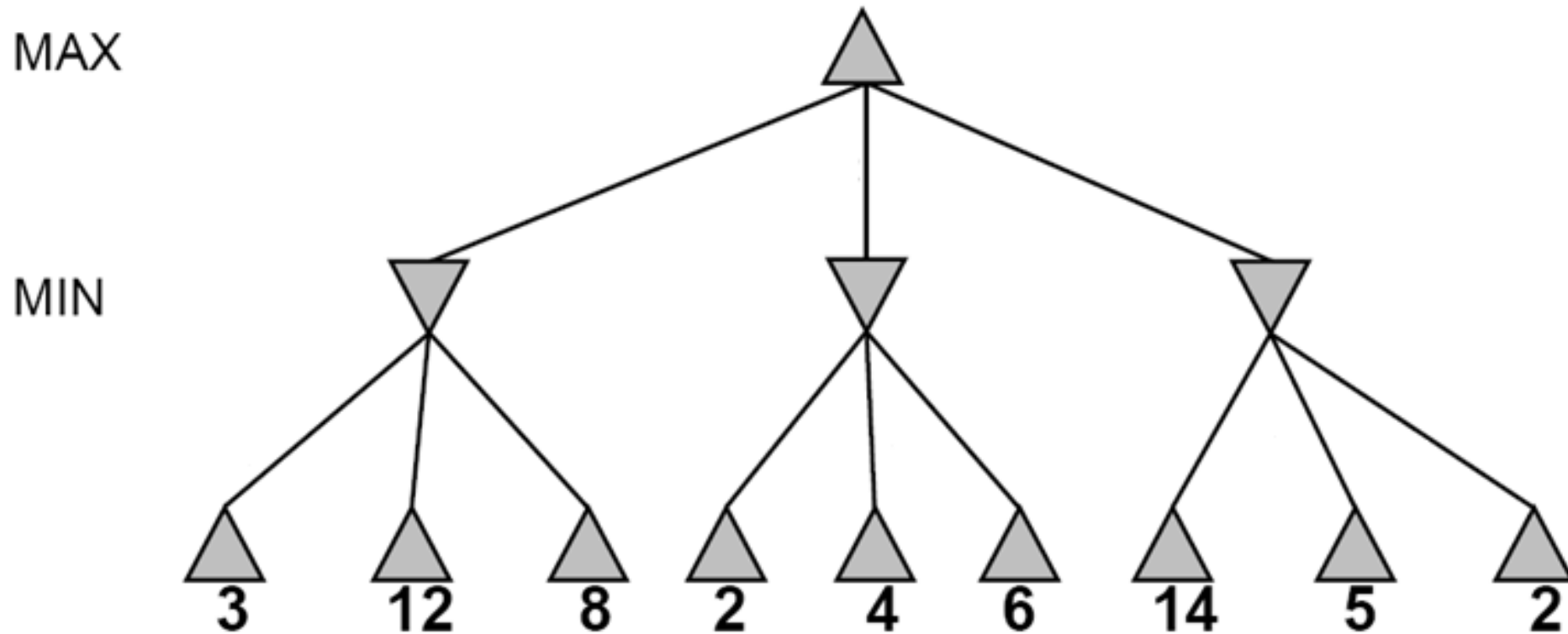
Alpha-Beta Pruning

- Traverse tree in depth-first order
- At **MAX** node n , **alpha(n)** = max value found so far
 - Alpha values start at $-\infty$ and only increase
- At **MIN** node n , **beta(n)** = min value found so far
 - Beta values start at $+\infty$ and only decrease
- **Beta cutoff:** stop search below MAX node N (i.e., don't examine more descendants) if $\alpha(N) \geq \beta(i)$ for some MIN node ancestor i of N
- **Alpha cutoff:** stop search below MIN node N if $\beta(N) \leq \alpha(i)$ for a MAX node ancestor i of N



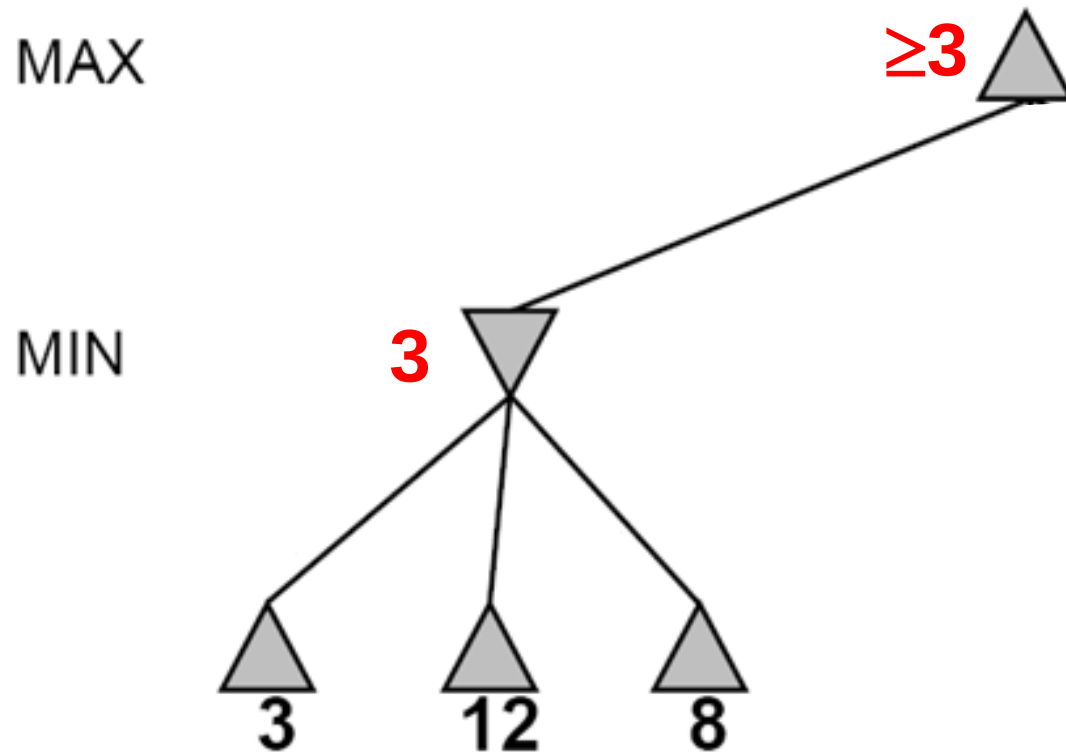
Alpha-beta pruning

It is possible to compute the exact minimax decision without expanding every node in the game tree



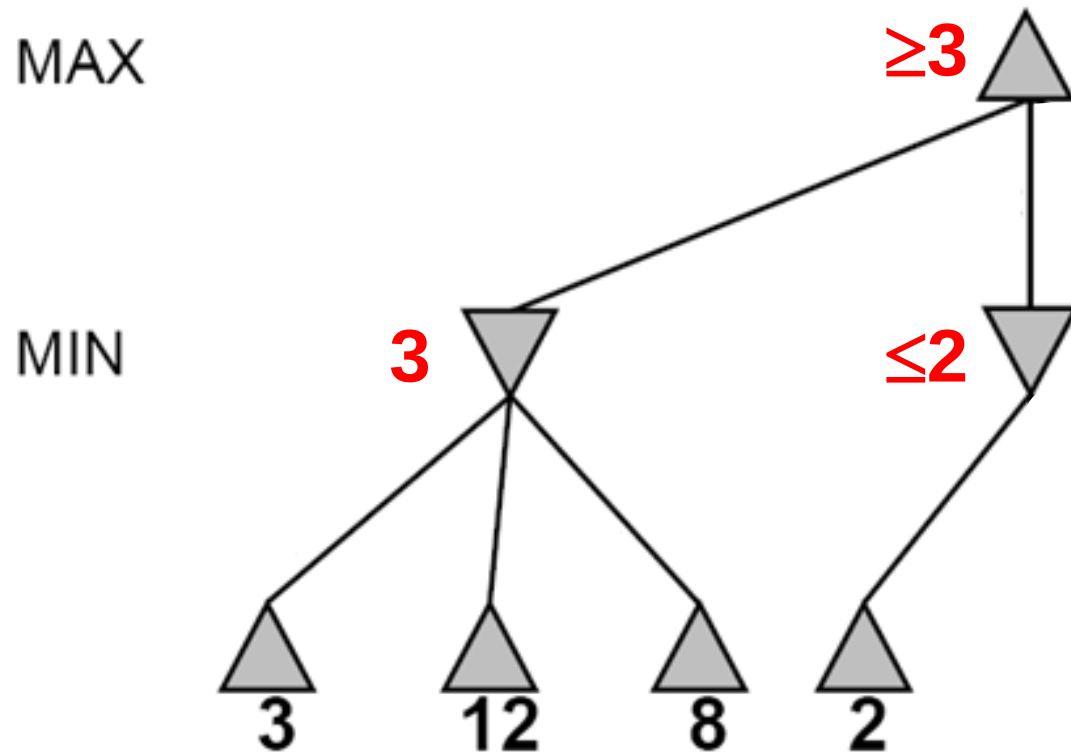
Alpha-beta pruning

It is possible to compute the exact minimax decision without expanding every node in the game tree



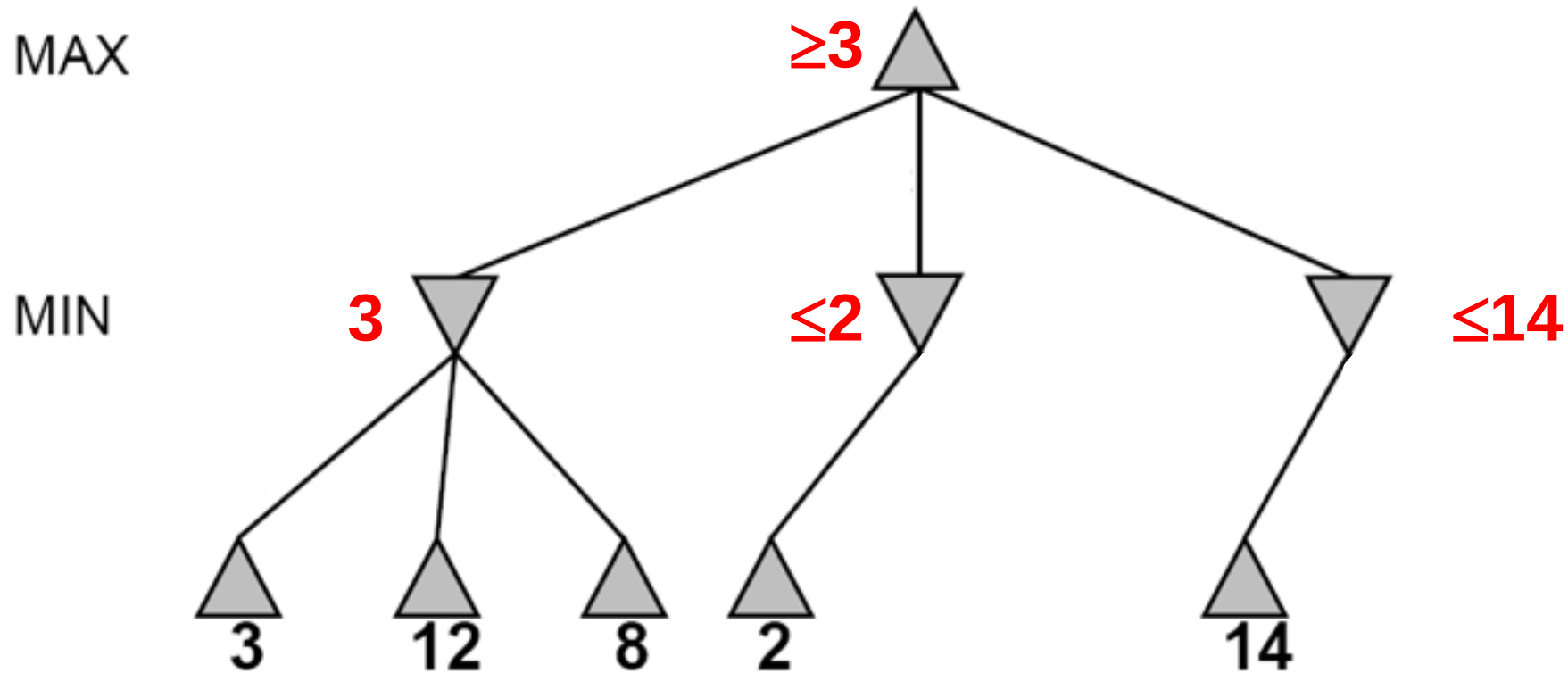
Alpha-beta pruning

It is possible to compute the exact minimax decision without expanding every node in the game tree



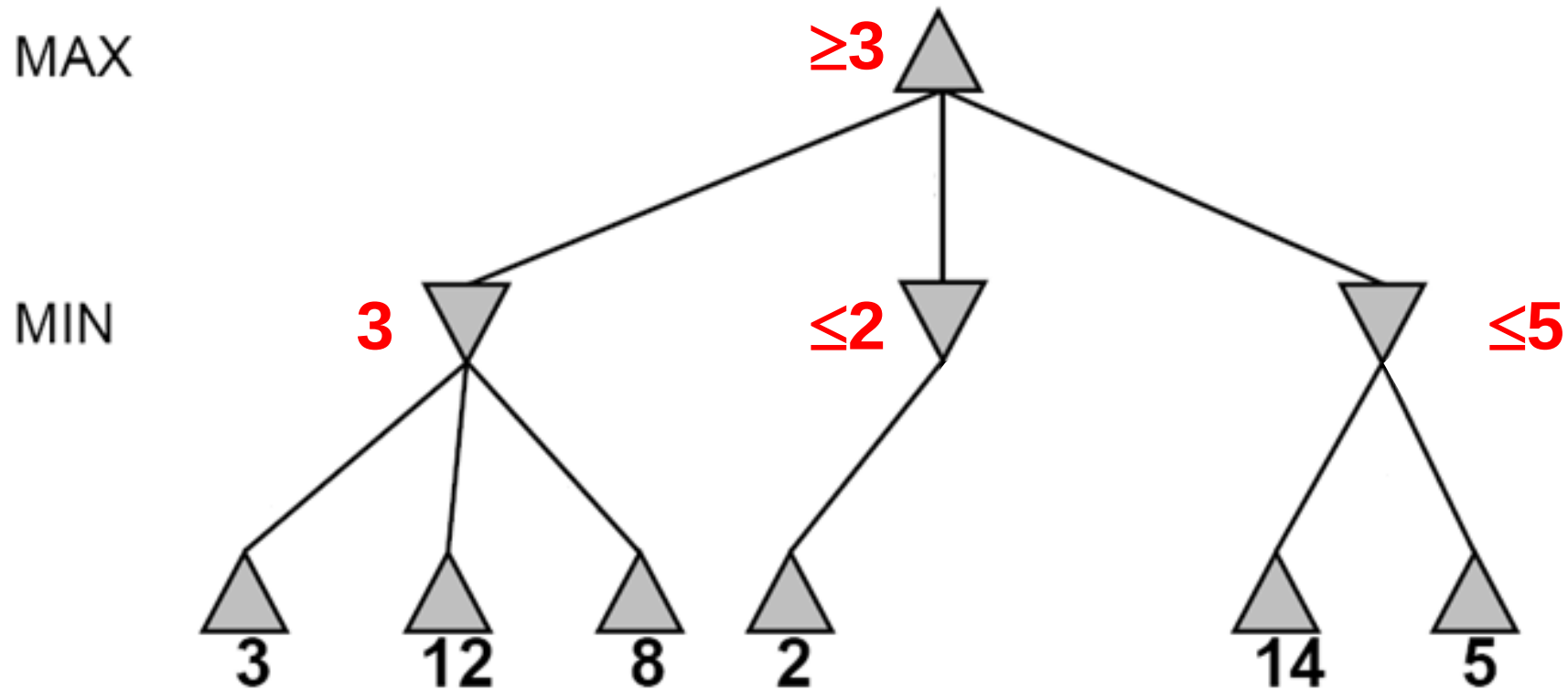
Alpha-beta pruning

It is possible to compute the exact minimax decision without expanding every node in the game tree



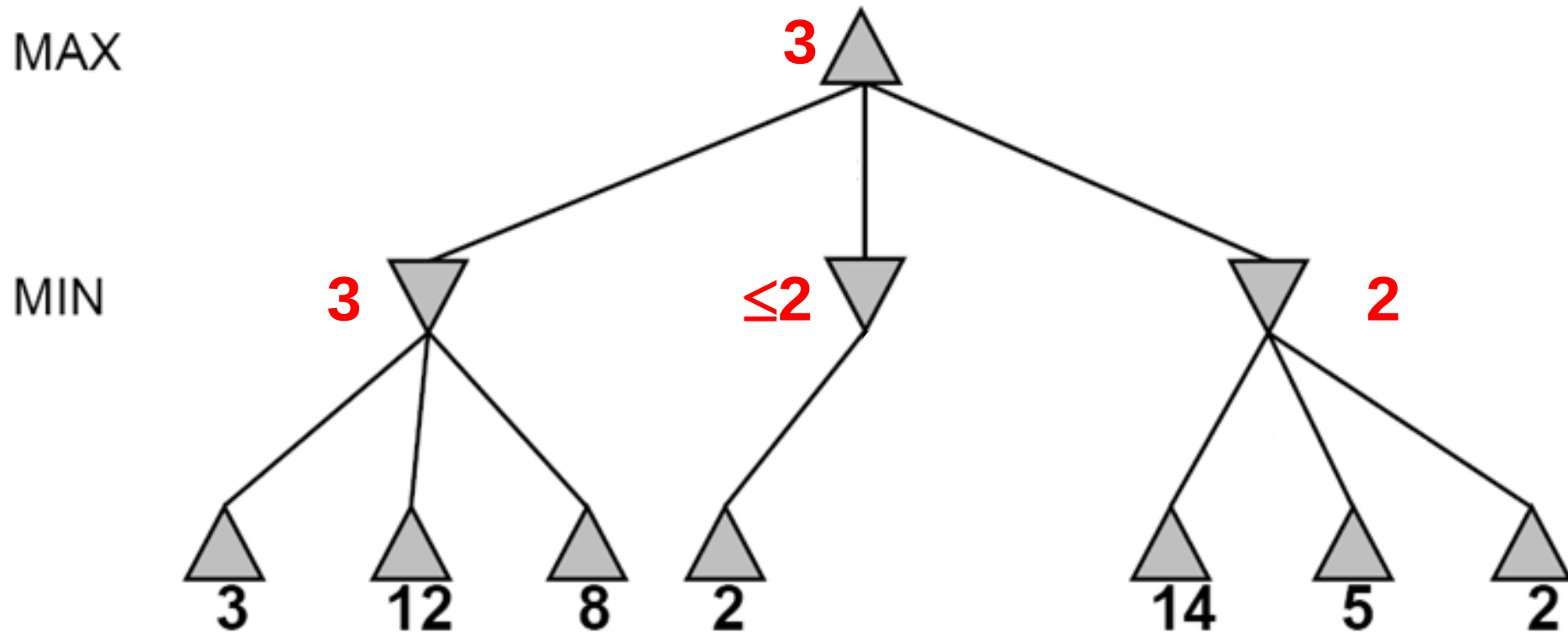
Alpha-beta pruning

It is possible to compute the exact minimax decision without expanding every node in the game tree

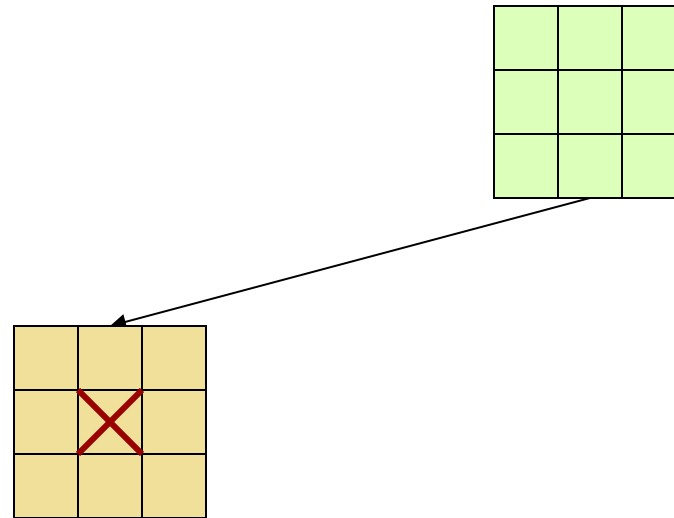


Alpha-beta pruning

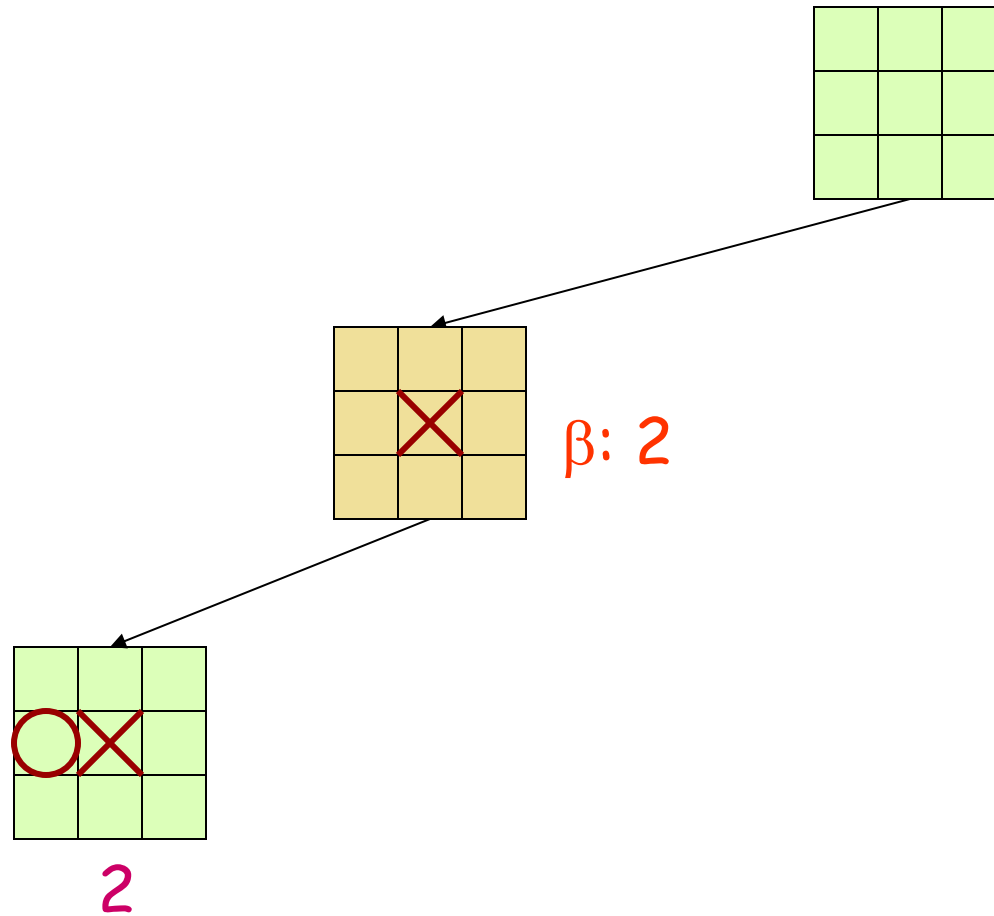
It is possible to compute the exact minimax decision without expanding every node in the game tree



Alpha-Beta Tic-Tac-Toe Example



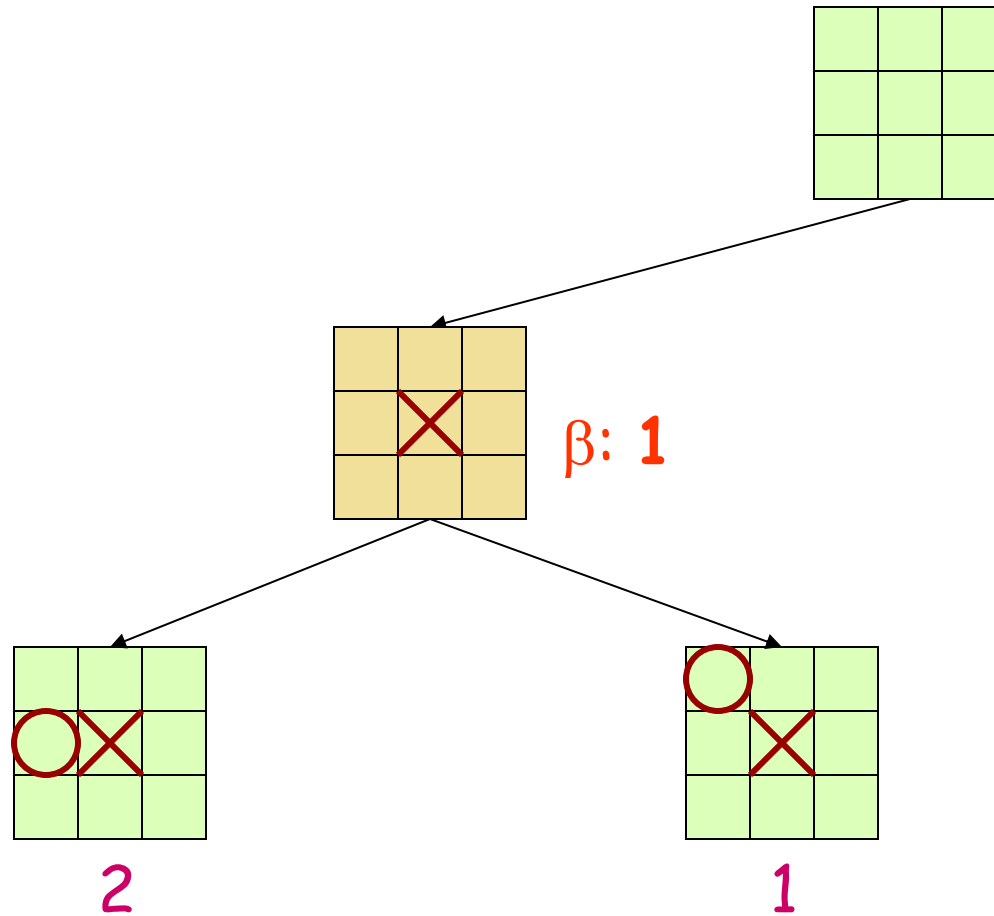
Alpha-Beta Tic-Tac-Toe Example



Utility = X's open lines –
O's open lines

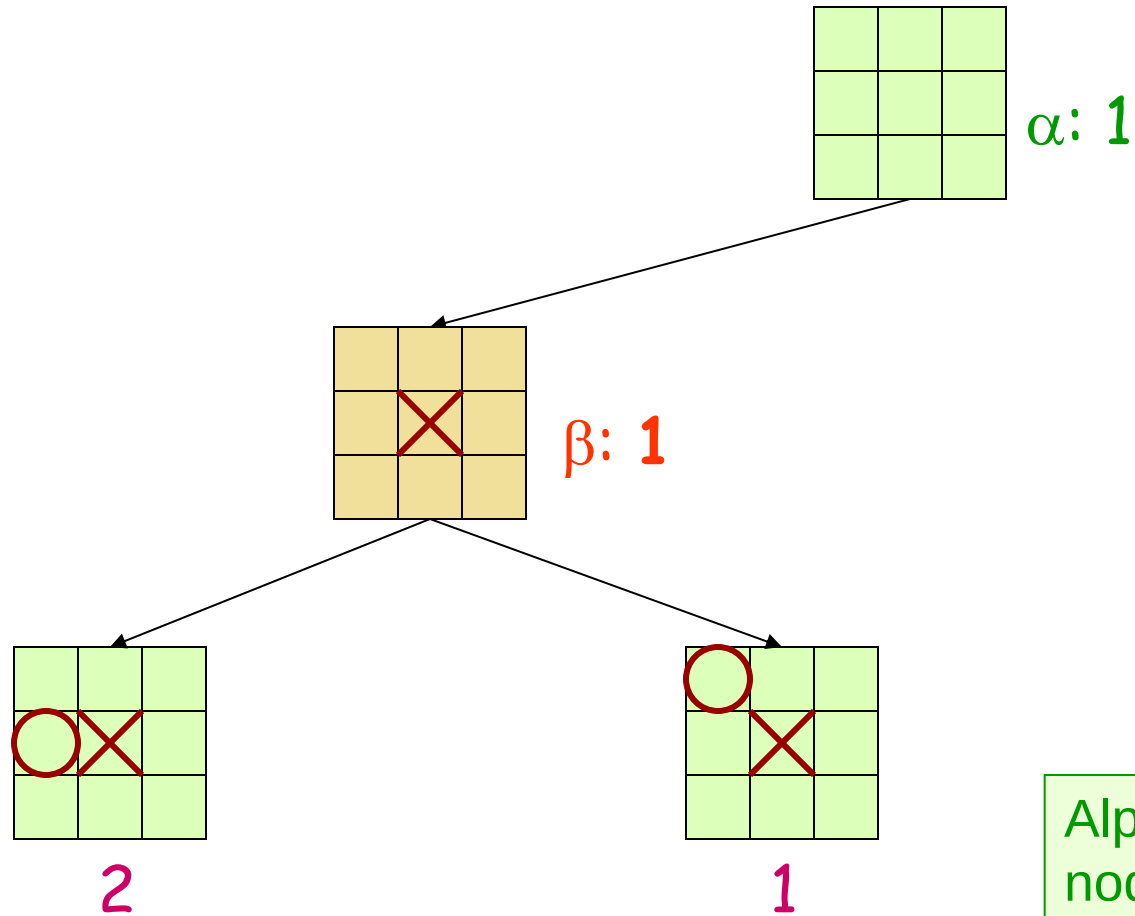
Beta value of a MIN
node is **upper** bound on
final backed-up value;
it can never increase

Alpha-Beta Tic-Tac-Toe Example



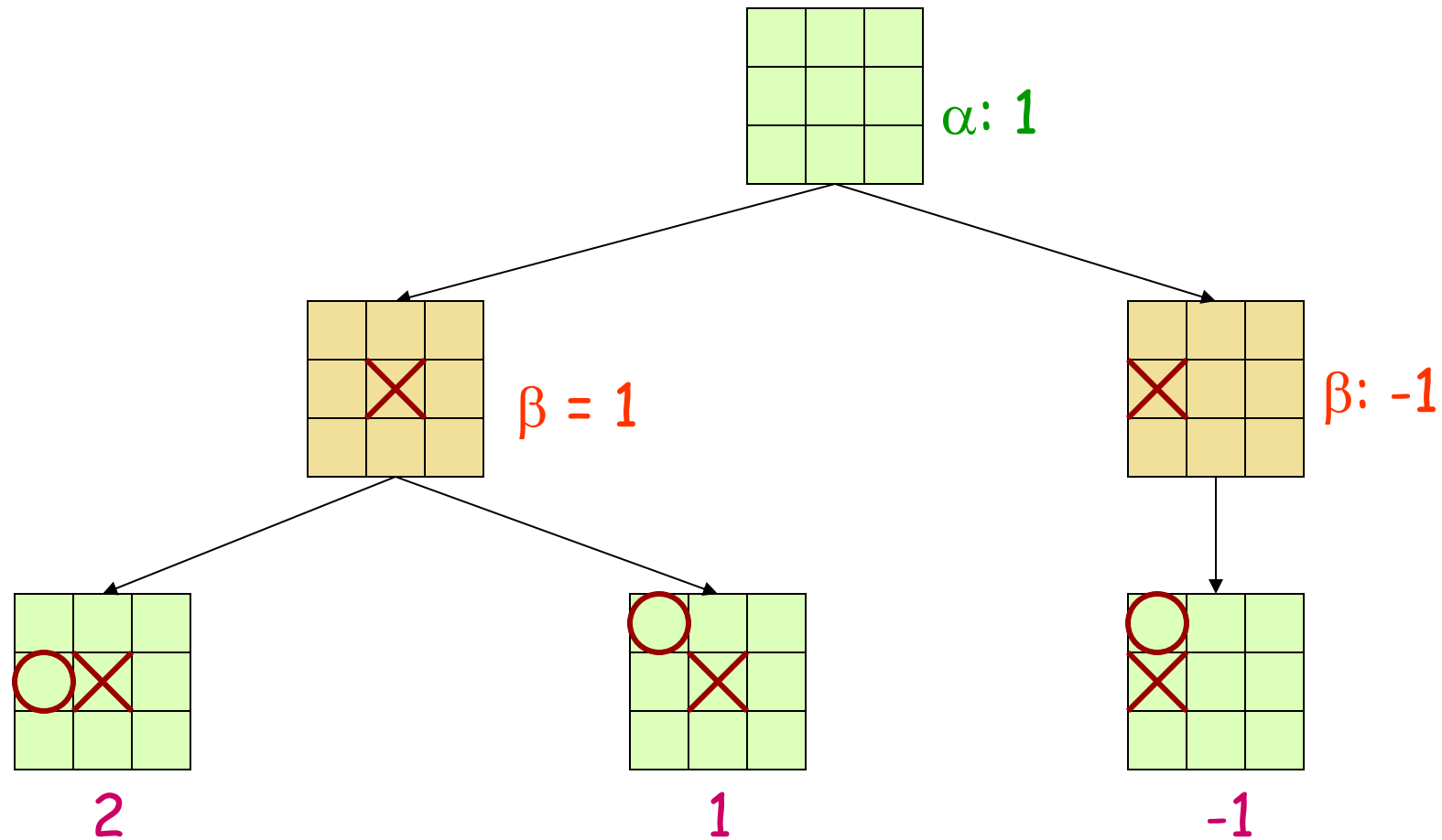
Beta value of a MIN node is **upper** bound on final backed-up value; it can never increase

Alpha-Beta Tic-Tac-Toe Example

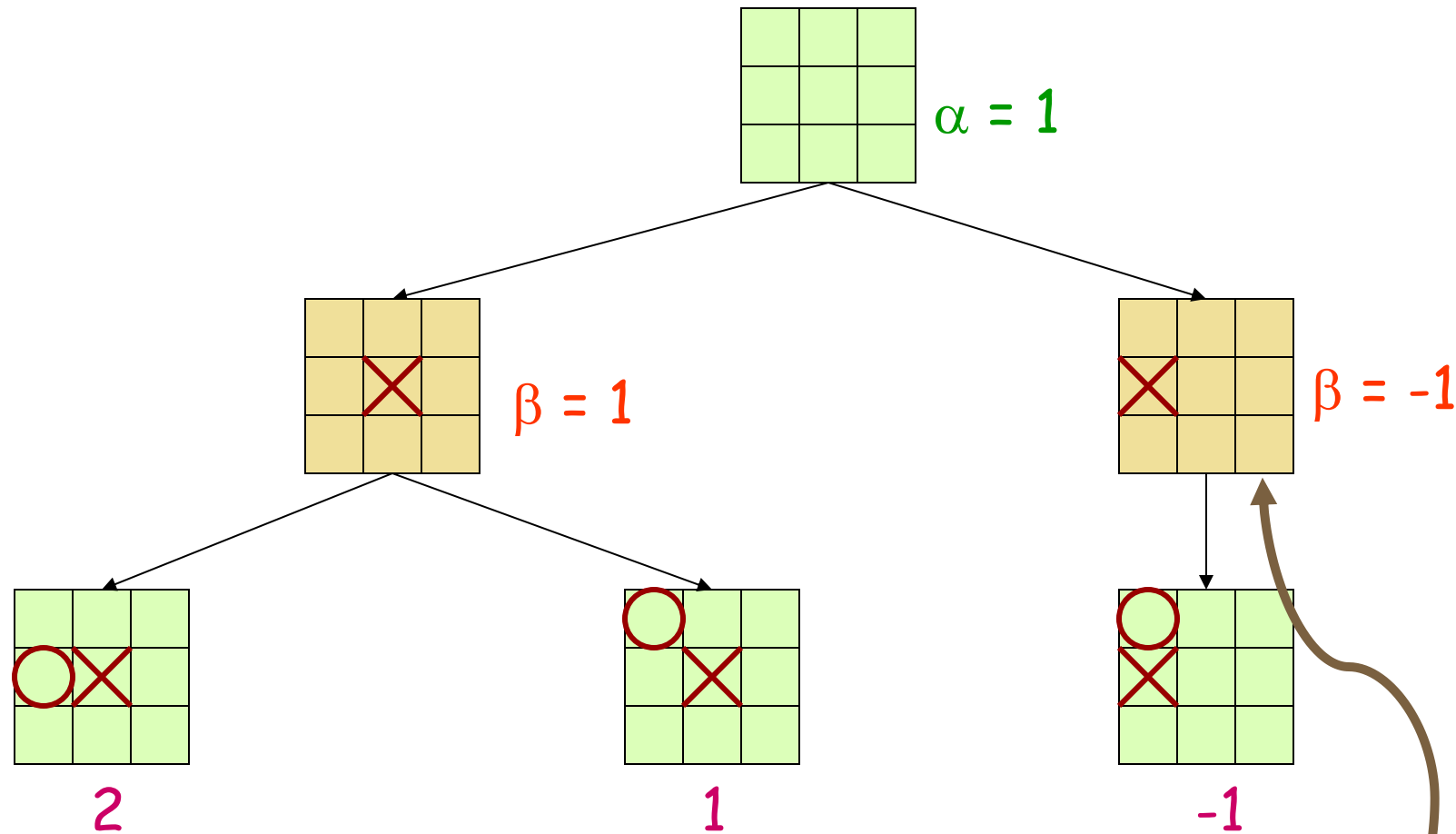


Alpha value of MAX node is **lower** bound on final backed-up value; it can never decrease

Alpha-Beta Tic-Tac-Toe Example

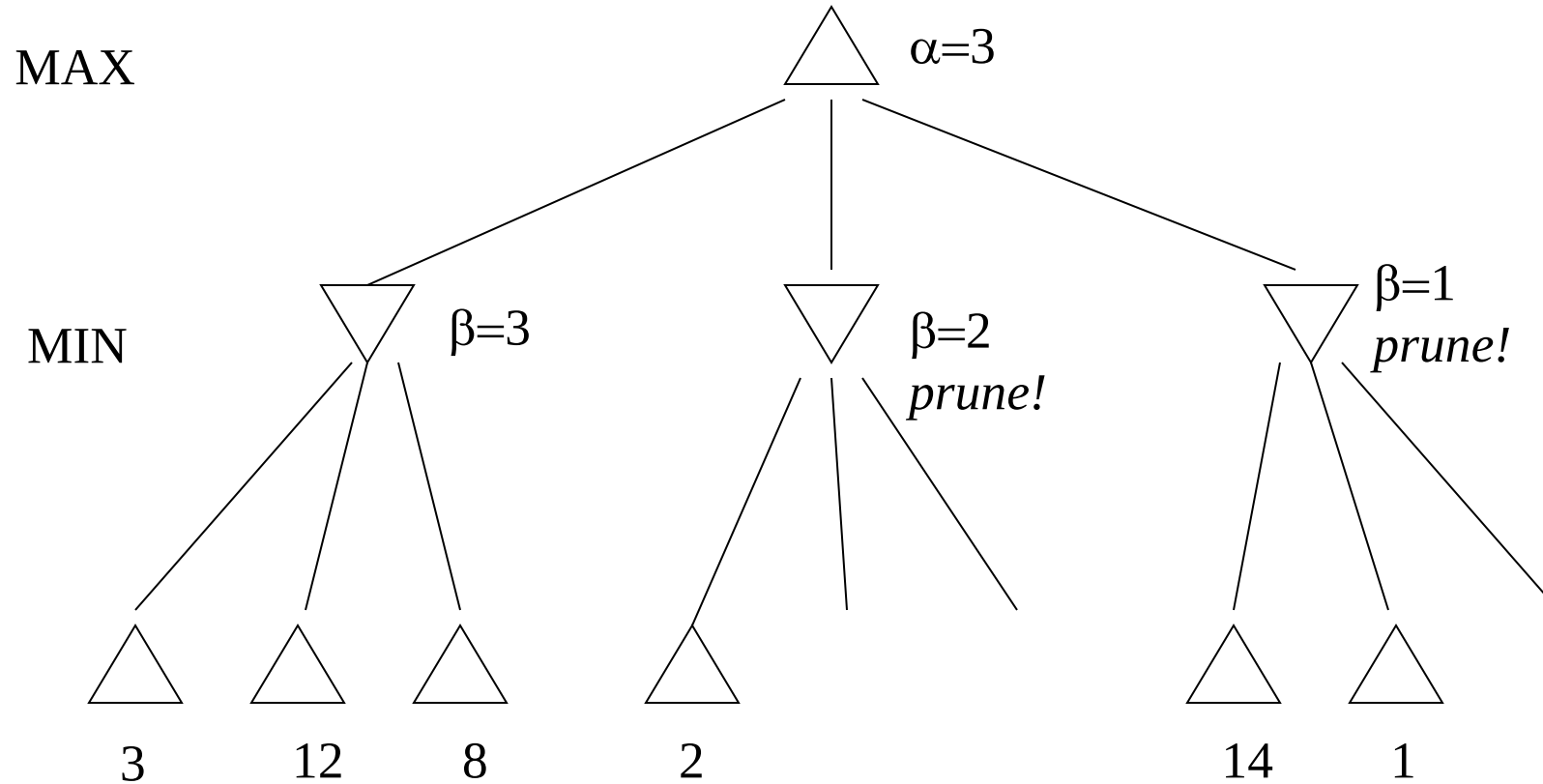


Alpha-Beta Tic-Tac-Toe Example



Discontinue search below a MIN node whose beta value $\leq$ alpha value of one of its MAX ancestors

Another alpha-beta example



Additional techniques

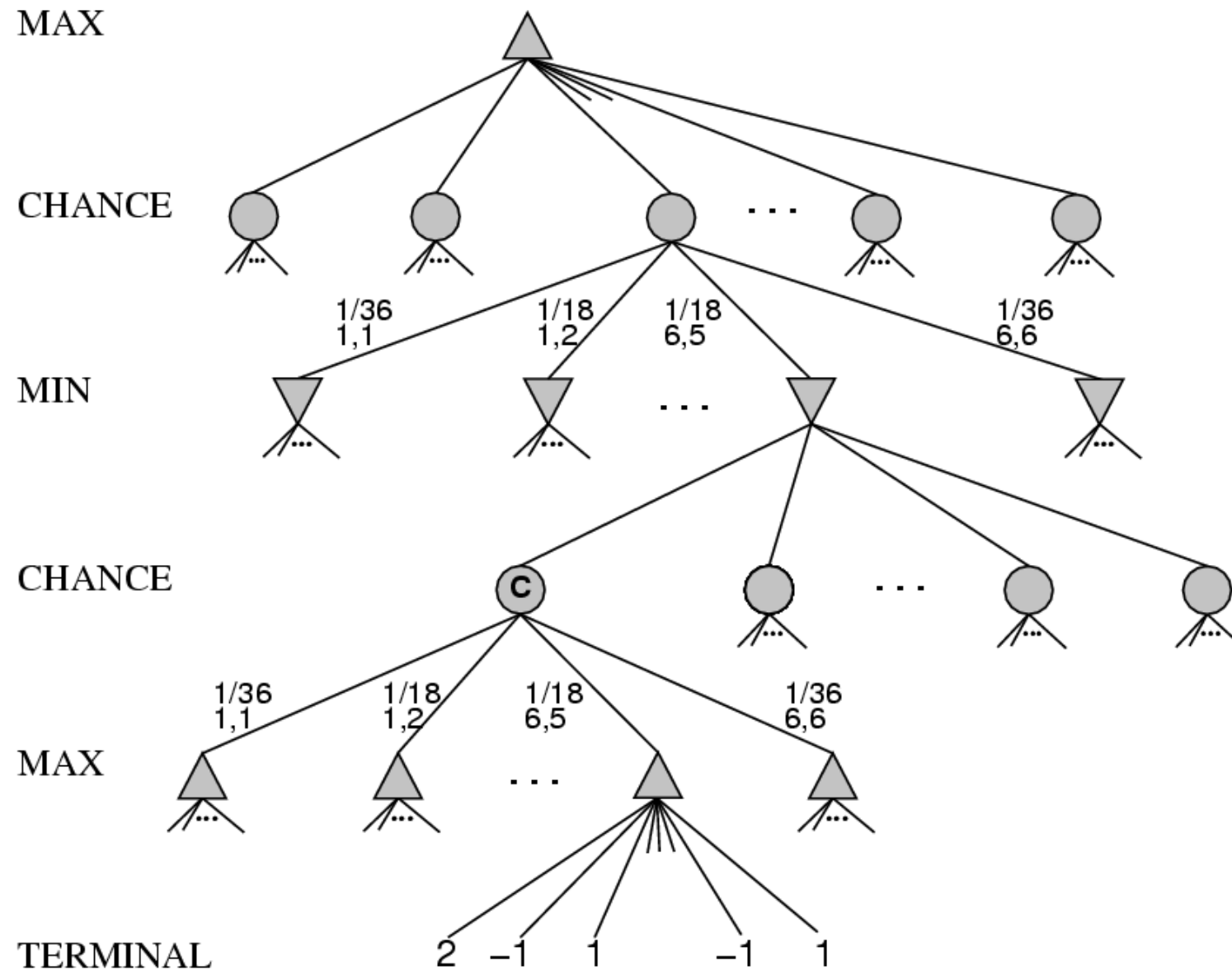
- **Transposition table** to store previously expanded states and conclusions
- **Forward pruning** to avoid considering all possible moves
- **Lookup tables** for opening moves and endgames

Games of chance

How to incorporate dice throwing into the game tree?



Games of chance



Games of chance

Expectiminimax: for chance nodes, average values weighted by the probability of each outcome

- Nasty branching factor, defining evaluation functions and pruning algorithms more difficult

Monte Carlo simulation: when you get to a chance node, simulate a large number of games with random dice rolls and use win percentage as evaluation function

- Can work well for games like Backgammon

Game playing algorithms today

Computers are better than humans

- **Checkers:** [solved in 2007](#)
- **Chess:** IBM Deep Blue defeated Kasparov in 1997

Computers are competitive with top human players

- **Backgammon:** [TD-Gammon system](#) used reinforcement learning to learn a good evaluation function
- **Bridge:** top systems use Monte Carlo simulation and alpha-beta search

Computers were not competitive, but now they are 😊

- **Go:** branching factor 361. Existing systems use Monte Carlo simulation and pattern databases. **AlphaGo's** algorithm uses a Monte Carlo tree search to find its moves based on knowledge previously "learned" by machine learning, specifically by an artificial neural network (a deep learning kind) by extensive training, both from human and computer play
<https://www.youtube.com/watch?v=g-dKXOlsf98>
- 2019: Google AI beats top human players at strategy game StarCraft II, [link](#)

Origins of game playing algorithms

- Ernst Zermelo (1912): Minimax algorithm
- Claude Shannon (1949): chess playing with evaluation function
- John McCarthy (1956): Alpha-beta search
- Arthur Samuel (1956): checkers program that learns its own evaluation function by playing against itself

**Thank you for
your attention**